BoilerCompass

Design Document

Team 7: Hamilton Wang, Jenny Chen, Joanna Lin, Mark Tseng, William Hou, Zoey Lu

Index

Purpose 3

Functional Requirements 3

Non-Functional Requirements 7

Design Outline 8

High-Level Overview 8

Overview Activity Diagram 6

Design Issues 10

Functional Issues 10

Non-Functional Issues 12

Design Details 14

Class Diagram 14

Description of Classes and Interaction Between Each Other 14

Sequence Diagram 17

- Create an Account Sequence Diagram 17
- Guest User Sequence Diagram 19
- Login Sequence Diagram 21
- Add Course Sequence Diagram 22
- Classlist bar and its uses 23
- See shared schedules within groups 24

Overview Navigation Flow Diagram 25

UI Mockup 26

Purpose:

Something that every motivated college student values is having a plan for their degree. In many departments at Purdue, students are provided a simple spreadsheet in which to manually form a plan of study for their time as an undergraduate. Even though myPurduePlan provides a platform for students to see courses they have taken and the requirements for graduation, there exists no easy way to visualize and validate a comprehensive plan of study that spans the duration of their college career. Our project, BoilerCompass, will allow all Purdue undergraduate students to effortlessly create and manage their degree plans through a user-friendly web application that uses verified information such as course prerequisites and degree requirements to aid students as they create plans, as well as to validate completed plans. Students will be able to share their schedules, and coordinate to enroll in courses together. Further, schedule owners can edit access permissions to authorize other users to copy the schedules as templates.

Instead of only providing insight into coursework already completed, or simply acting as a calendar application, BoilerCompass should be a tool to allow students to plan the future for their degree with confidence. The application will aggregate facts about Purdue courses from various sources in addition to the same information regarding degree and credit requirements that myPurduePlan gives, and present this information to the user in the same platform used to plan their degree, from their upcoming semester to their final semester before graduation. Additionally, Purdue academic advisors will be able to create templates on BoilerCompass with the courses they commonly recommend to the students in the department they work in so that they can work more efficiently. Similarly, students will be able to seamlessly communicate their semester plans to their advisor by sharing their plans directly with their advisor.

Functional Requirements:

1. **User account creation and management**
    a. As a user (both students and advisors)…
        i. I would like to create a personalized account with an email and password the first time I use the website so that I can get access to the service. This account creation includes verifying my email address.
        ii. I would like to log into my account with a Purdue email and password.
        iii. I would like to be able to reset my password if I forget it.
        iv. I would like to change my profile information, such as password and dark/light mode preference, on the settings page.
        v. I would like to change my profile picture on my profile page.
        vi. I would like to have the option to delete my account whenever I choose to.
        vii. I would like to enable two-step authentication for improved account security.
    b. As a student…

i. I would like to begin using BoilerCompass without having to create an account yet, so that I can easily begin using the app.
ii. I would like to easily export and import the entirety of my user data, so that I may migrate to a different device if I do not want to use BoilerCompass with an account.
iii. I would like to choose my major, minor, degree focus, and expected graduation date.
iv. I would like to log out from my profile and also delete my account when I graduate.
v. I would like to manage my personal profile privacy settings to enable or disable public viewing of certain details on my profile page.

c. As an advisor…
i. I would like to access the students' profile pages so that I can see details of their schedules.
ii. I would like to access my own profile page including name, profile picture, and department. I would also like to log out from the profile page.

2. User interactions and connections

a. As a user…
i. I would like to be able to view the profiles of myself as well as other users, and view the details they choose to make public, such as their name, profile picture, email, major, and shared schedules and templates.
ii. I would like to be able to save a schedule (either template or others' schedules that you have permission to see) for later reference.
iii. I would like to be able to manage my saved schedules and templates created by other users for later use while planning.
iv. I would like to block and unblock other students so that I can exclude particular students from being able to view details on my profile that are publicly viewable by others.
v. I would like to be able to send requests to other users to give them permission to access my schedule, and I would like to be able to accept requests to access others' schedules.

b. As a student…
i. I would like to create a group and share my schedule with other members.
ii. I would like to manage my groups by adding and removing members after a group is created.
iii. As a student, I would like to review the comments from my advisor in the plans and accept them or not, and or reply to them.

c. As an advisor…
i. I would like to see a list of my students, and click on each of the students to see more information about each student. The information should be as much as the students can see (aside from personal settings)
ii. As I look at the students' information and their plans, I would like to make suggestions and comments on their schedule.

3. Degree planning

a. As a student…
    i. I would like to see a descriptive graph that clearly delineates the order of the courses that should be taken.
    ii. I would like to manually check which requirements I have completed, and which one I will complete when choosing future courses.
    iii. I would like to see a summary of each semester, including the total credit hours, as well as total credit hours throughout the whole college life.
    iv. I would like to upload my academic history from myPurdue Plan and have my degree progress automatically populated. (If time permits and conditions allow)
    v. I would like to see syllabuses from the past for courses that I am considering taking. (If time permits and conditions allow)
    vi. I would like to be able to see a list of courses (under each category) and choose the ones I want to add to the specific semester on my schedule.
    vii. I would like to be able to click on a specific course to see its information, including credit hours, course descriptions, and prerequisites.
    viii. I would like to be able to search for the courses I want to add.
    ix. I would like to be able to set a filter for courses and see the resulting courses based on my filters.
    x. I would like to be able to have the capability to easily add new courses to an already created schedule by simply dragging and dropping these courses into it.
    xi. I would like to be able to view saved schedules and manage a plan simultaneously, so that I may copy parts of the schedule or simply look at it for reference when editing my plan.
    xii. I would like to receive a warning if a semester schedule I've made fails to meet certain criteria, such as the minimum/maximum credit hours needed/allowed, time conflicts, missing prerequisites, or contains fall-through credits.
    xiii. I would like to see data about grades achieved by students during past offerings of a particular course so that I can make an informed decision about the effect a course I take might have on my GPA.
    xiv. I would like to know if a class is waivable with placement exams.
    xv. I would like to view all sessions of lectures and labs for a specific class in a certain semester.
    xvi. I would like to have a separate page for a specific semester, in which I can add the sessions I want and generate a weekly schedule.
    xvii. When creating a plan for a specific semester but course sessions have not been chosen, I would like to have auto-populated schedules that pick sessions based on starting time, ending time, or the most compact arrangement.
    xviii. I would like to create and delete customized blocks that I can add to my weekly schedule.

xix. For each block I created, I would like to change the name, duration, and color of the block.
xx. I would like to be able to see a recommended schedule for my major, and the schedule will be able to be rearranged as I put in my transfer credits.
xxi. I would like to experiment with changing/adding different majors and see the path of graduation to help me visualize this hypothetical idea.
xxii. I would like to be able to tag/mark classes that I am not sure if I want to take or not and access them later.
xxiii. I would like to be able to enter my grades for the courses I already took and the expected (or goal) grades for future courses and see the resulting GPA.

b. As an advisor…
   i. I would like to create a planning template for different majors prepopulated with suggested courses and timelines for students to use.
   ii. I would like to compare the schedules of all my current students to give better suggestions.
   iii. I would like to save the schedules of all my past students to understand how students' preferences may have changed.
   iv. I would like to track and monitor students' progress toward degree completion through a dashboard.

4. Miscellaneous (customization, data import/export)

a. As a user…
   i. I would like to download my plan in formats such as PDF or CSV.
   ii. I would like to switch between dark and light themes on the settings page on the settings page.
   iii. I would like to customize the text style, size, and the background color of the app to make reading and navigating more comfortable for me..

b. As a student…
   i. As a user, I would like to seamlessly/effortlessly transfer my course plans from my account into the UniTime course request system with a single action so that I only need to plan it once and have it in my request form.
   ii. I would like to choose a template created by an advisor or student that contains all the suggested courses and timelines, and populate my schedule with its contents.

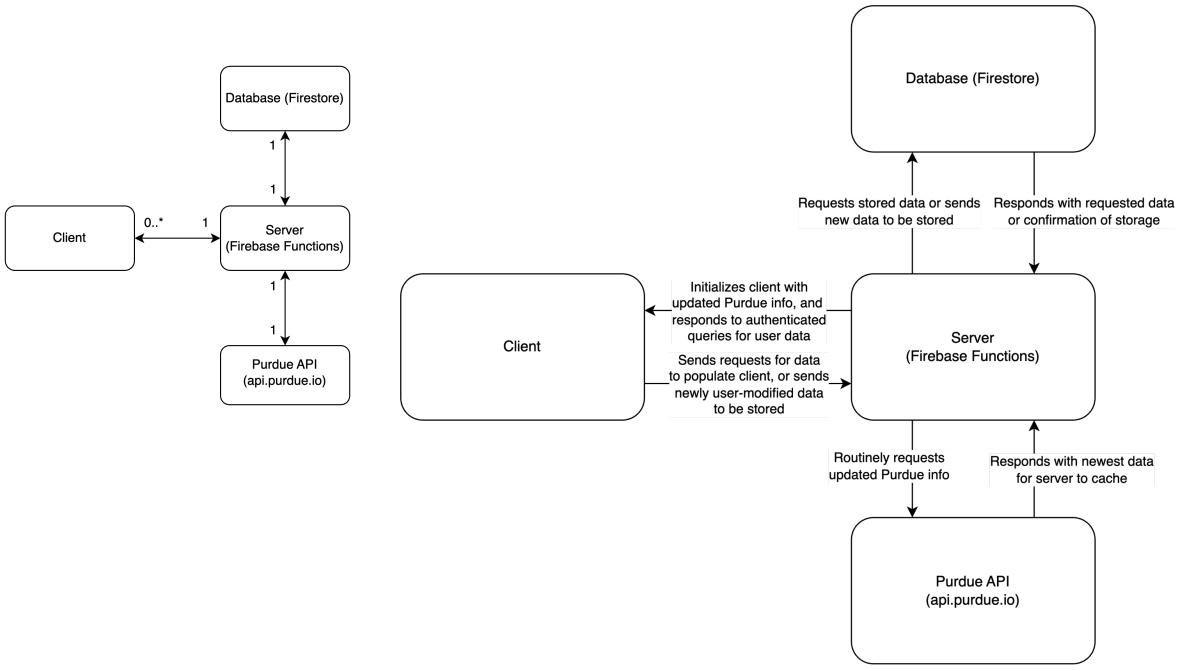
Non-Functional Requirements:

1. Website capacity:
    a. The website should be able to function properly under the capacity of 50,000 Purdue students and advisors using it at the same time.
2. Data privacy:
    a. The website should be able to protect all students' personal schedule data from being accessed by outsiders without the students' permission. This will be done via encryption and authentication/privacy controls using Firebase.
    b. The database of the website should be safely protected and prevent unauthorized access to users' information.
    c. The verification process for re-setting passwords and verifying passwords should be secure and reliable.
3. Uptime and load times:
    a. The website should be able to locate and present a student's schedule under 5 seconds.
    b. All Purdue students and advisors verified by their Purdue email should have access to the website 24 hours a day.

Design Outline

High-Level Overview

BoilerCompass will be a web application that allows Purdue students to plan their undergraduate degrees and share their plans with advisors and classmates. The application will use a client-server backend structure. Each React.js user Client will invoke Firebase Functions, which will act as the server, using a REST API. The server will query a Firebase Firestore database to access data, as well as external APIs to retrieve relevant data regarding Purdue courses and degrees. The server will respond to all client queries, and provide responses containing data when data is requested.



1. React.js Client
    - The React javascript Client will run in the browser of the user. It will be the access point to the BoilerCompass system.
    - The Client will communicate with the server to retrieve up-to-date data regarding the user's own account (degree plans, user connections) as well as Purdue data.
    - Users will use the Client's planning interface to create new plans, manage existing plans, and share plans with other users.
    - The Client will be responsible for validating plans at the time they are made, and sending saved plans to the server.
2. Firebase Functions Server

- A collection of functions running on Firebase Functions will be triggered as user Clients make requests to their HTTPS endpoints.
- The functions will query a single central Firestore database to store or retrieve data.
- The functions will respond to Client queries after performing the requested action (POST data to be stored or GET stored data) with a response containing the result of the requested operation, and data if it was requested.
- The server will routinely query external APIs for Purdue course and degree data, and cache it in the Firestore database for later retrieval when Clients request it.

3. Firebase Firestore Database
    - Firestore is a NoSQL database service provided by Firebase that will be queried via a Javascript API.
    - Data is stored persistently in Documents of key-value pairs, and Documents are stored in Collections.
    - A Collection of Documents containing users' details will be queried by the server when user data is needed.
    - A separate Collection for user plans will be queried as users manage and view their degree plans.
    - One single collection will contain cached Purdue data that every Client will query to use in validation.
4. Purdue.io API
    - Purdue.io API is an external API that uses web pages exposed publicly by myPurdue to collect Purdue course availability and schedule data.
    - Course prerequisite and degree requirements info is also accessible and retrievable through myPurdue
    - Purdue.io API will be queried routinely to retrieve up-to-date data that will be used by Clients to validate user plans.

Design Issues

Functional Issues:

1. Should the users be able to use our platform without registering for an account?
    - Option 1: Users must create an account before using the website.
    - Option 2: Users can continue without creating an account.

    Choice: Option 2

    Justification: By allowing the users to continue without creating an account, we lower the requirement for them to use our service, and this can encourage more people to try out our product. Also, this can give the users some time to try navigating through the application and consider if they want to continue using it. Forcing users to create an account is not suitable because users who do not wish for their data to be stored remotely by our services may choose to continue using the application without an account.
2. What email address domain can be used to create an account?

    - Option 1: Only Purdue email
    - Option 2: Both personal email and Purdue email
    - Option 3: Purdue career account

    Choice: Option 1

    Justification: Allowing people to create an account using outside email domains can make it hard for advisors and students to verify the identity of people trying to add them. Since our platform includes functionalities such as creating groups and sharing schedules, restricting to the Purdue email domain can prevent outside spam and enhance the security of data. Since we do not have access to the Purdue career account login system, the combination of limiting all users to utilize their Purdue emails, manually checking the email domain, and sending verification emails is the optimal solution for the project.

3. Should guest users be able to choose between student and advisor?
    - Option 1: Guest users can choose between a guest or a student
    - Option 2: Guest users can only be students

    Choice: Option 2

    Justification: The reason why guest users can only be students is because it doesn't really make sense for guest users to be advisors. Since guest users does not have to

verify their email, they should not be able to make suggestions to others' schedules or view degree progress of students. When advisors use this program, it's just for them to see and suggest changes on a student's schedule. Additionally, the overwhelming majority of people trying out this website will be using it to create temporary schedules, so it's better if there's lower overhead in doing so.

4. When should the users be asked to input profile information, including role (student or advisor), expected start/end date (students only), dark/light mode preference, etc.?

    - Option 1: Users input the profile information along with the account creation process (when they enter name, email, and password to create an account)
    - Option 2: Create a separate "profile setup" stage that goes after the basic account creation stage (email, name, and password) to input profile information
    - Option 3: Let the users to input their information in their settings page once they log in

    Choice: Option 2

    Justification: We decided to go for option 2 for a couple of reasons. First, because both normal users (with account) and guest users (without account) will need to enter their role and profile information (so that we know how to set up the schedule for them), creating a separate "profile page" will allow us to re-use the same code for both normal users and guest users. Secondly, during account creation process, we need to verify the user's email (sending them an email with a link to verify) in order to proceed further, and if we ask for their profile information before we can even verify their email, an overhead for both the user side and the system side will be created. Having a profile setup page after the account registration has completed will have a smoother work flow. Therefore, we chose option 2 over option 1. For option 3, since we need to set up the schedule based on users' profile information, asking about their information later will result in having to change the schedule setup again after their input, which is not necessary and inefficient.

5. Should users be able to chat in the groups?
    - Option 1: Yes. besides sharing schedules, users can also chat in the groups.
    - Option 2: No. Groups are only for sharing schedules.

    Choice: Option 1

    Justification: As a degree planning platform, we believe the main purpose of groups is for users to share schedules and chatting function is not needed. Furthermore, since our platform involves advisors and students, we believe it is better to keep discussions related to degree planning formal, through school email or meetings, instead of typing in the group.

Non-Functional Issues:

1. What web service should we use?
    - Option 1: Firebase
    - Option 2: AWS
    - Option 3: Microsoft Azure

    Choice: Firebase

    Justification: The number one reason why we chose Firebase is because we have team members who have experience working in Firebase. In addition, we chose to use a web service like Firebase that handles both the server and database because it simplifies our code structure, since we won't have to write our own server that interfaces with some database, and use another language like Python.

2. How should we temporarily store guest users' data?
    - Option 1: Cookies
    - Option 2: Users' computers own local storage

    Choice: Option 2

    Justification: Given that our main service is a degree-long planner, the temporary storage would need to store unlimited lists of courses for semesters, requiring a certain amount of storage. Since cookies are stored in a single string and are limited to merely 4KB, we have determined that cookies are not suitable for our application both in terms of storage structure and usable memory. Hence, we will utilize the user's computers' local storage for guest users.

3. How should we get the degree requirements and course prerequisites data?
    - Option 1: Request data from Purdue (in the form of either database or formatted files)
    - Option 2: Manually gather information online and input the formatted data into our application

    Choice: Ideally Option 1 (If no resources are available, we go for Option 2)

    Justification: Importing data directly from Purdue will be much more efficient, and we can quickly create the data system on a large scale, including all the majors, minors, and prerequisites. Gathering information manually will take a lot of time and may include inaccurate data. Furthermore, since Purdue already has systems such as UniTime and myPurdue, it is reasonable to believe that they already have the data we

need in their database. manually collecting the same information again on our own does not make sense.

4. How should we verify the user's email address?

   - Option 1: Only check the domain (@purdue.edu)
   - Option 2: Try to get a list of Purdue email users and check if the email is in the list
   - Option 3: Check the domain first, and then send the user an email to verify if the email actually exists and belong to the user

   Choice: Option 3

   Justification: We decided to go with option 3 because it adds extra protection for the users, and it also reduces possible overhead for our database. For extra protection, if a person tries to use other students' email addresses to register, choosing option 1 or 2 will allow that person to use/steal the identity of others. This can even make the true owner unable to register with their email address. In terms of overhead, if we do not check if the email address actually exists (if we choose option 1), there may be an infinite number of junk accounts in our database if people create accounts with a non-existent email address with the Purdue email domain.

5. How often should the server query Purdue API to retrieve updated course and degree info?

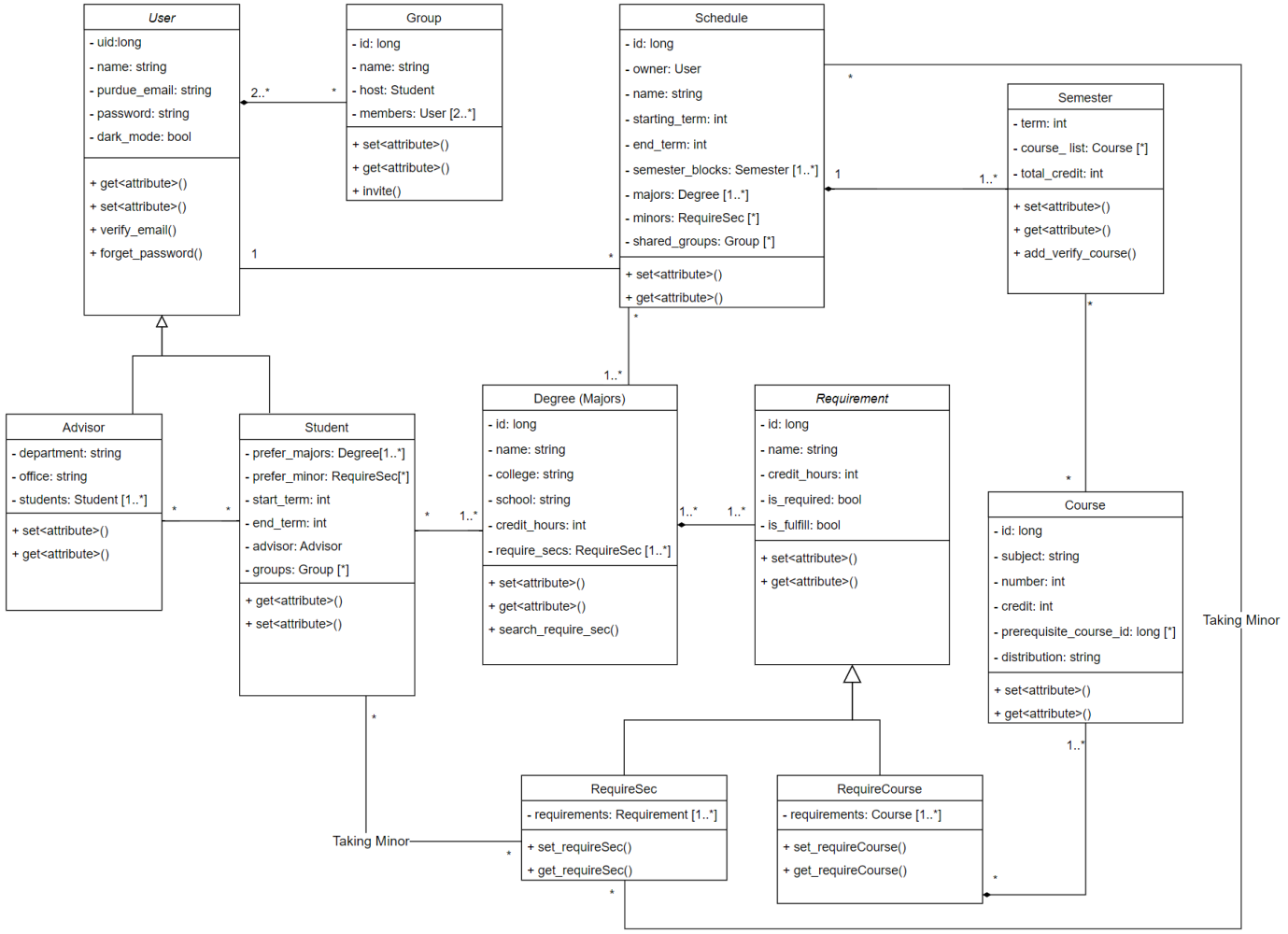
   - Option 1: As often as possible (every time a user Client requests it from the server)
   - Option 2: Routinely, every day
   - Option 3: Once per term (each semester, and summer and winter sessions)

   Choice: Option 2

   Justification: Although it may be the case that course details are unlikely to change from the beginning of each term, a single query per 24 hours is infrequent enough to have almost no impact on the operation of Purdue API as a service, but allows us to cater to any possible update in a timely manner. Allowing clients to query Purdue API each and every time information is needed will cause unnecessary traffic to the service. Furthermore, there is no set date that Purdue promises to make course info for the next term available, so a daily update will be sufficient to poll for info.

Design Details

Class Diagram



User

- A new user object (either student or advisor) will be created when they first create an account.
- Each user will fill in their email and password for login purposes, as well as their name when setting up their profile.
- Each user is either a student or an advisor. As students and advisors share common attributes, the user class serves as an interface for the two classes.
- A user can create multiple schedules, which are degree-long planners, and courses to specific semesters through dragging and dropping.

Student

- A student object is created when the user first creates an account and is verified as a student.

- When setting up the profile, each student will fill in additional information, including the starting term of their degree, the expected ending term of their degree, preferred majors, and preferred minors.
- Each student is associated with their preferred majors and minors (which are represented by Degrees and RequireSec correspondingly) as a default setting when creating a new schedule.
- Each student must take at least 1 major and multiple, including 0, minors.
- Each student can add courses to specific semesters in a schedule through drag and drop.
- Each student can create groups, at least 2 students, with other students to share each others' schedules.

Advisor

- An advisor object is created when the user first creates an account and is verified as an advisor.
- When setting up their profile, each advisor will fill in their department and office location as additional information.
- Once linked, each advisor can view their students' schedules and provide suggestions.
- Each advisor can create a planning template for different majors prepopulated with suggested courses and their respective semesters for students to reference.

Group

- A group object is created when a student sends a request to their friends to form a group and they also accept the invitation. It's also created when a student forms a group with their advisor.
- Each group contains an id, name, host (student that created the group), and members.
- Within the group, members can share their schedules with each other.

Schedule

- A schedule object is created when a user first creates an account and goes to the homepage or when a user creates a new schedule. The initial schedule will be empty until the user adds courses.
- Each schedule contains an id, the owner, who it belongs to, name, starting term, ending term of the plan, and semesters that range from the starting term to the ending term.
- Each schedule is associated with set majors and minors.
- Each schedule can be downloaded.
- Each schedule can be shared in multiple groups, including 0 when the student does not have a group.

Semester

- Semester objects are created when a schedule is created.
- Each semester contains multiple courses and only belongs to one specific schedule.
- When a course is added/removed, the total credit hours of the semester will be updated and verifications for prerequisites are performed.

Degree

- A degree object is created when a student declares a major and when a new schedule is created.

- Each student or schedule has at least one degree (major).
- Each degree contains multiple requirement sections (e.g. University Core Curriculum, Major in Computer Science, Other Departmental Course Requirements…etc) that a student must fulfill to graduate.
- A degree can be applied to multiple schedules.

Requirement

- A Requirement object relating to a specific major is created when a student declares a major or a new schedule with a specific major is created.
- Requirement is an interface for RequireSec (Require Section) and Requirement.
- Requirements are satisfied when a student takes RequireCourses, which come from RequireSec, in their schedule.

RequireSec (required sections)

- RequireSec objects relating to a specific major are created when a student declares a major.
- There can be multiple RequireSec objects within a requirement. For example, in the technical writing and presentation requirement, the RequireSec objects are: 1. Technical Writing. 2. Technical Presentation. 3. Technical Writing and Presentation Combination. Students with this requirement can choose to take courses from RequireSec 1 and 2 or just RequireSec 3 to satisfy the technical writing and presentation requirements.

RequireCourse

- RequireCourse objects relating to a RequireSec and a specific major are created when a student declares a major or a new schedule with a specific major is created.
- There are multiple RequireCourse objects within a RequireSec. For example, the RequireSec of the University Core Curriculum will contain RequireCourse objects, such as courses within UCC:Written Communication and courses within UCC:Oral Communication …etc.
- Each RequireCourse object can be found as a Course object, but a Course object isn't necessarily a RequireCourse.

Course

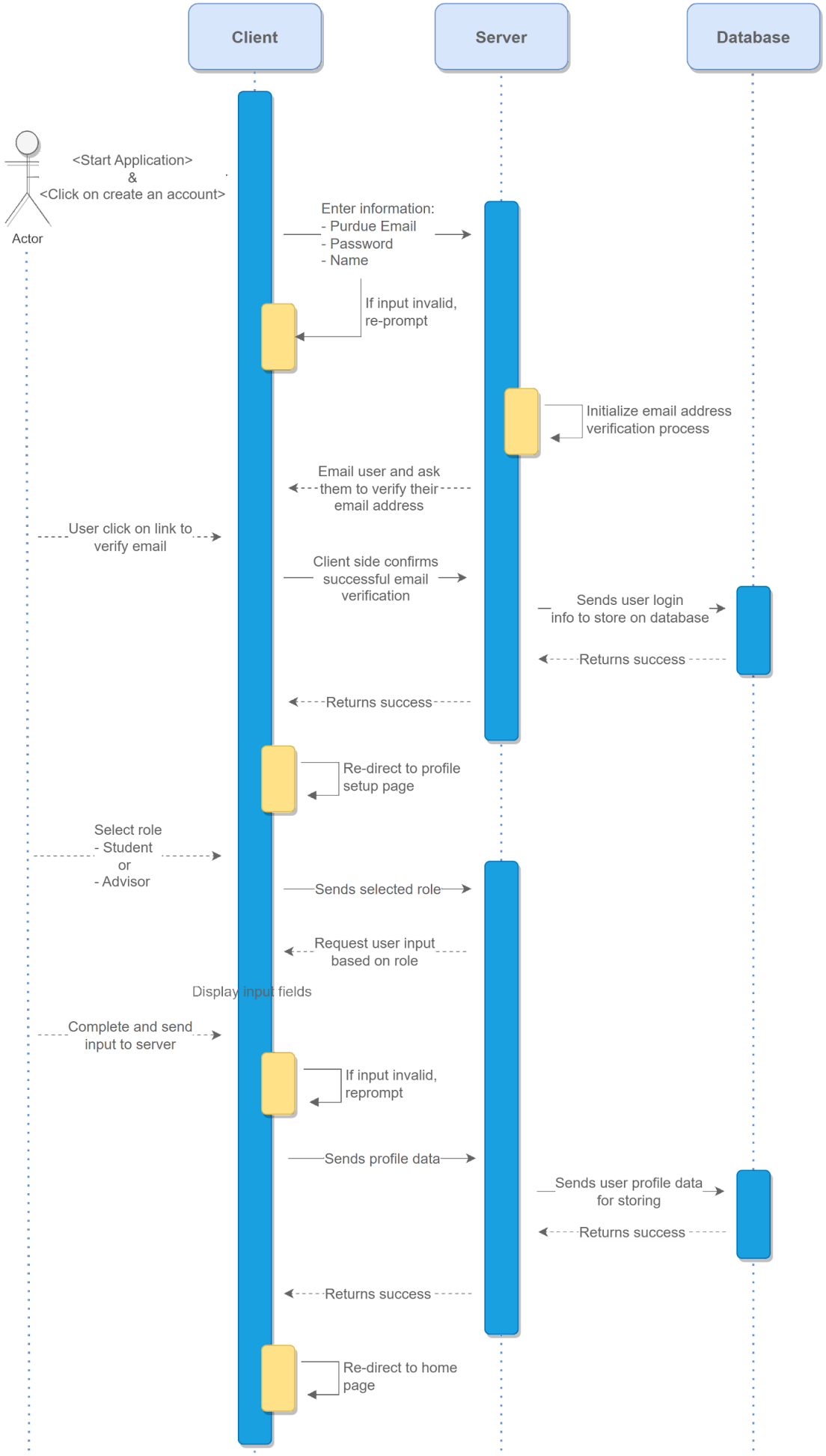
- Every course object is created and added to the database when the website first starts running.
- Each course contains an id, name, credits, and prerequisites.
- Each course can be added to a student's schedule and their information can be seen by advisors and students.
- A course object can be part of RequireCourse that helps satisfy the student's degree requirements, or it can also just be a fall-through course that doesn't contribute to the student's degree requirements.

Sequence Diagram

Below are the sequence diagrams for important events in our application, including creating an account, guest user, user login, adding courses, searching and filtering courses, and seeing group shared schedules. They show how a series of actions and interactions occur between the client (user and frontend), server, and database. The client initiates actions/requests and sends information to the server, the server then processes accordingly and interacts with the database to retrieve and store data. Requested data or indication of successful storage will then be returned to the user, enabling the UI to update the display for users.

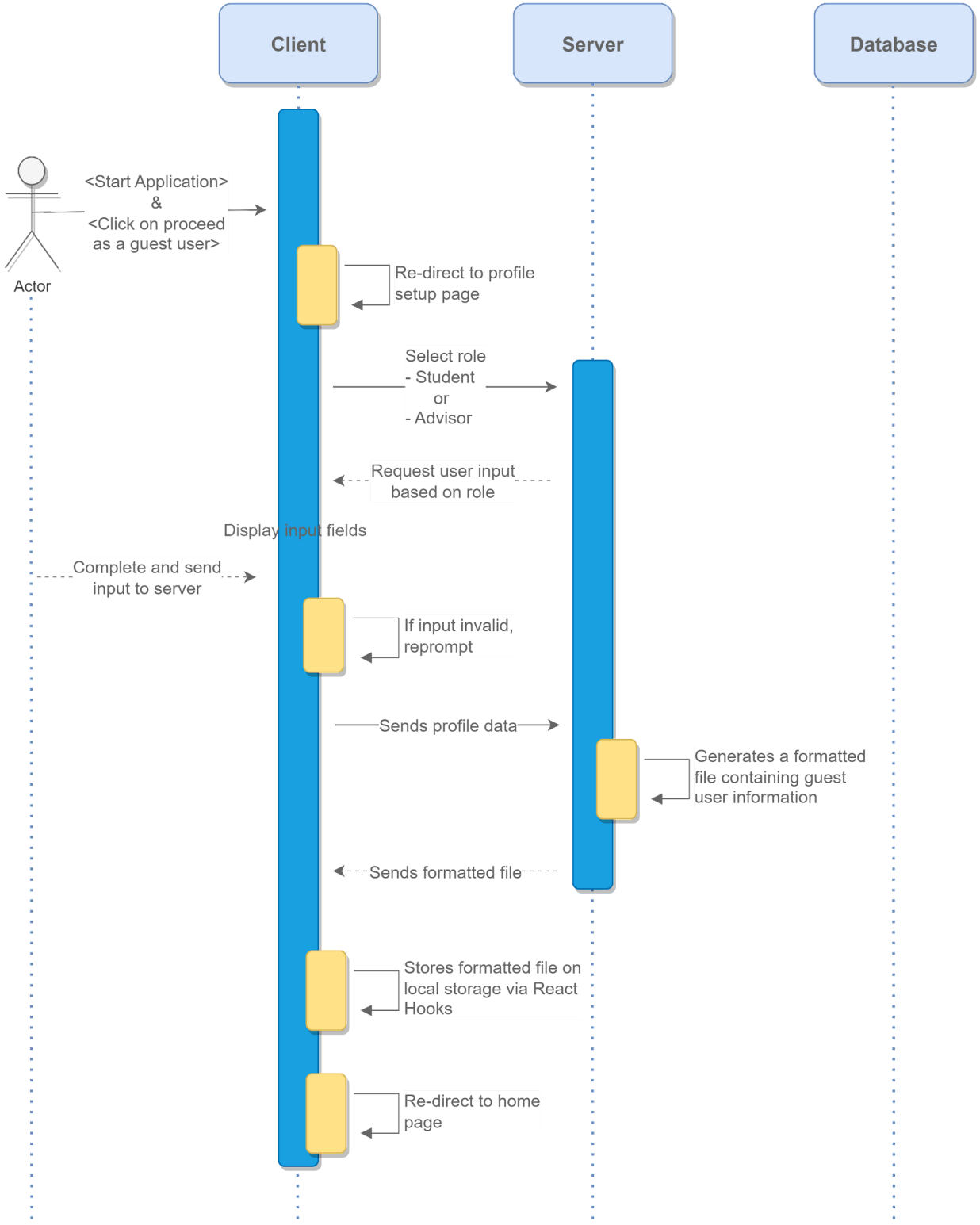
- **Create an Account Sequence Diagram**

  The below sequence diagram shows that when the user chooses to create an account, they will be inputting information (purdue email, password, and name) to the server. If the input is invalid, the client side will re-prompt the user for input. Once the server receives the information, it will initialize an email verification process, in which an email with a verification link is sent to the user for them to click on. This doesn't involve the database because we do not want to store the user's information until the email is verified (otherwise our database will be populated with potentially junk data (invalid email)). Once the user email is verified, we can then proceed to storing user information to the database, which completes the account creation process. After that, the user is then directed to the profile page, in which they input their profile data that gets sent to the server and eventually to the database for storing. After the process is completed, the server returns success (this is only an indication to the client that the process has been completed), and the client platform can then redirect the page to the homepage for further user actions.


Client
Server
Database
Actor
<Start Application>
&
<Click on create an account>
Enter information:
- Purdue Email
- Password
- Name
If input invalid,
re-prompt
Initialize email address
verification process
Email user and ask
them to verify their
email address
User click on link to
verify email
Client side confirms
successful email
verification
Sends user login
info to store on database
Returns success
Returns success
Re-direct to profile
setup page
Select role
- Student
or
- Advisor
Sends selected role
Request user input
based on role
Display input fields
Complete and send
input to server
If input invalid,
reprompt
Sends profile data
Sends user profile data
for storing
Returns success
Returns success
Re-direct to home
page

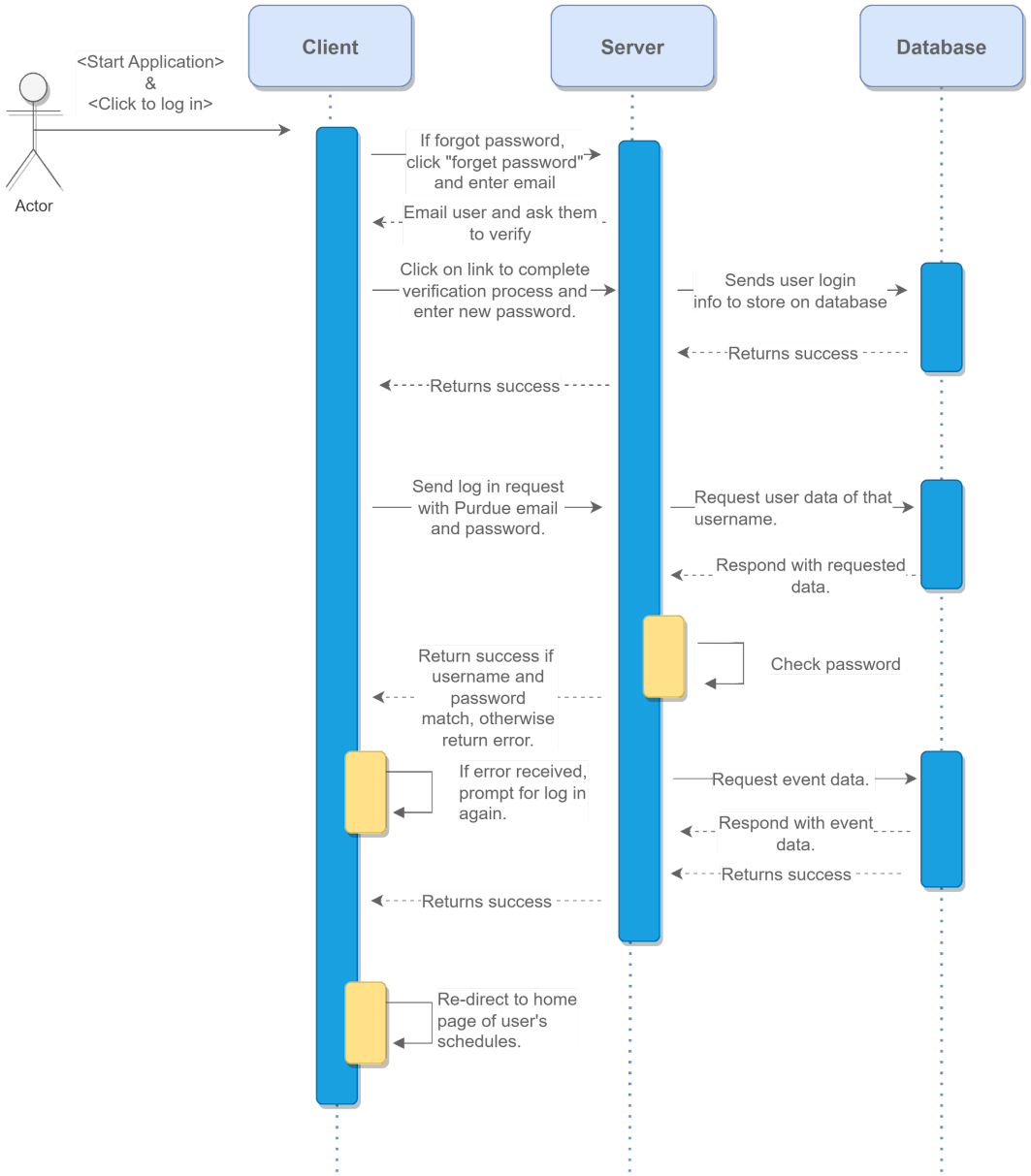
- **Guest User Sequence Diagram**

  The below sequence diagram shows a series of events that happen after the user chooses to proceed as a guest. This process is similar to the normal account creation, but it is different in (1) guest user does not have to go through the first account creation (inputting name, email, and password) (2) we store the guest user's information in their local storage instead of the database. In the diagram, after the user has inputted profile information, which gets sent to the server, the server then generates a formatted file with that information and sends it back to the client side, the client side then stores the file into the user's local storage. This will be done with the help of React Hook, which is already part of the client platform (React) (since it is not an external tool, we only put it as client side in the diagram to reduce confusion).


Client
Server
Database
Actor
<Start Application>
&
<Click on proceed
as a guest user>
Re-direct to profile
setup page
Select role
- Student
or
- Advisor
Request user input
based on role
Display input fields
Complete and send
input to server
If input invalid,
reprompt
Sends profile data
Generates a formatted
file containing guest
user information
Sends formatted file
Stores formatted file on
local storage via React
Hooks
Re-direct to home
page

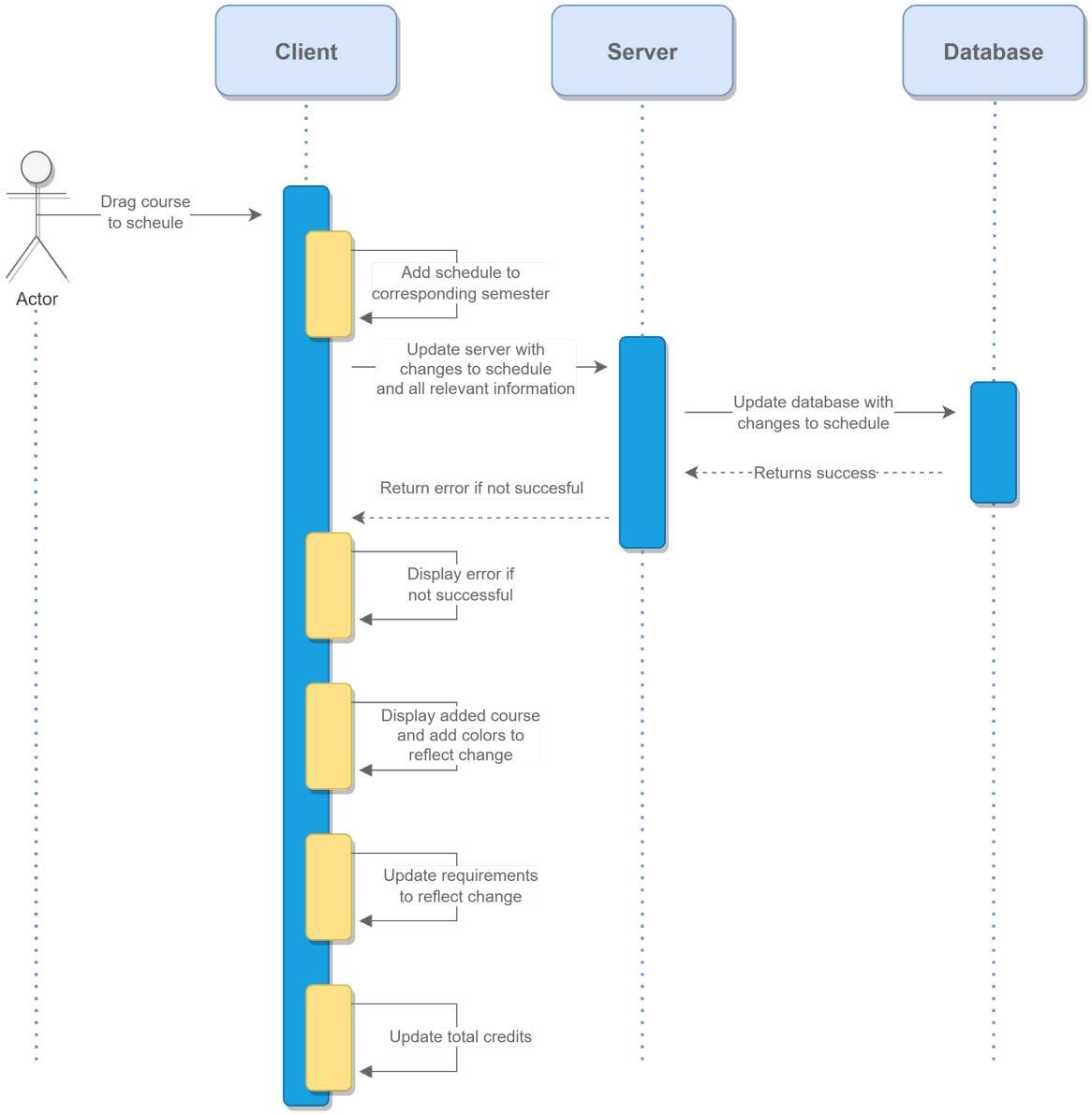
- **Login Sequence Diagram**

  The below sequence diagram shows that when the user logs in to their profile, they have two options: one is that they will input their account information (email, password) to the server. Then the server goes to the database and retrieves the information based on the user's email. Then the server checks whether the email and password from input match the ones that were already stored. If they match then the server returns success, allowing the user to go into their homepage. If not, then the server returns an error and will cause the user to enter their information again. The second option is to do "forget password", where the user verifies their email first, then enters a new password to be saved. After that then the user can log in again using their newly set-up password. Eventually, the user is directed to their homepage.



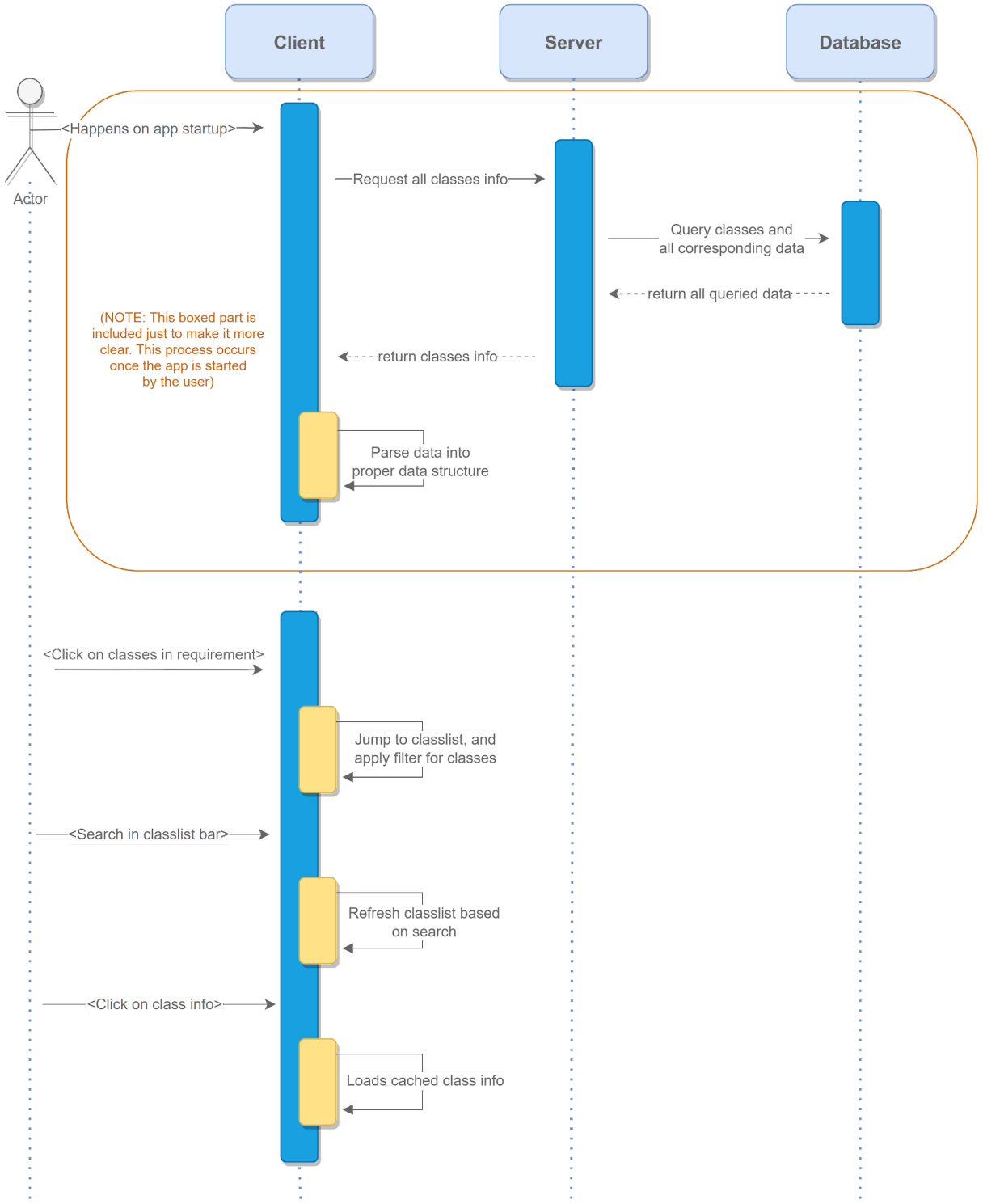
- **Add Course Sequence Diagram**

  This describes the flow of the user adding a course to the schedule. The user can drag a course from the courselist to the schedule section. The course is added to the semester, and the client updates the server and updates the database with the changes. The client will handle any errors and display them properly. If successful, the new course will be displayed, with the correct colors and ui to reflect its completeness, or if the prerequisites are satisfied. The requirements list will also update its ui/colors to include the newly added class.



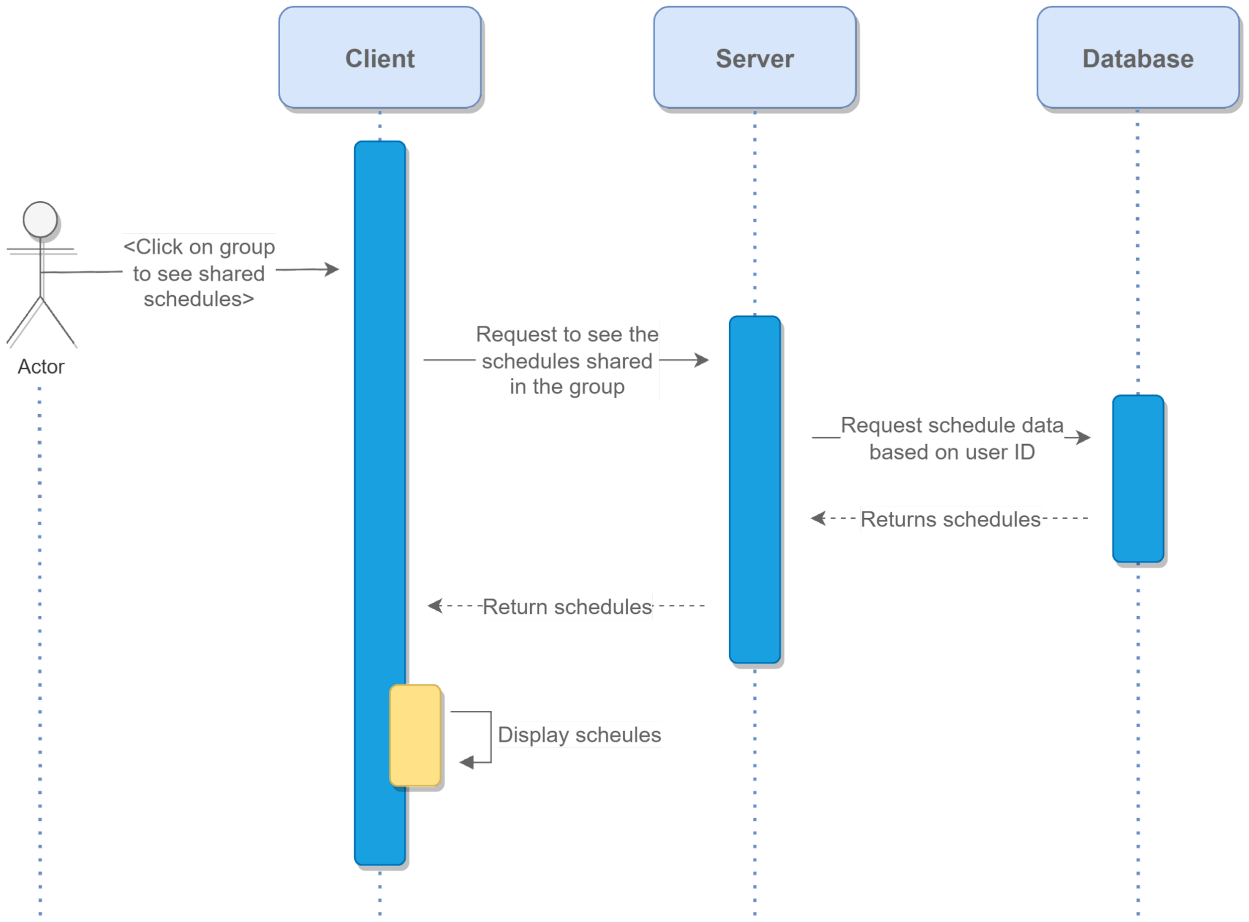
- **Classlist bar and its uses**

  This describes the flow and uses of the classlist sidebar. The list of classes should always be able to be displayed, so all the classes and info should be loaded into the client on startup. After that, it's only a matter of how the classlist is displayed with filters and the search bar. Users can also filter the classlist if they are directed from the requirements, and there should also be additional info for each of the class that can be shown.



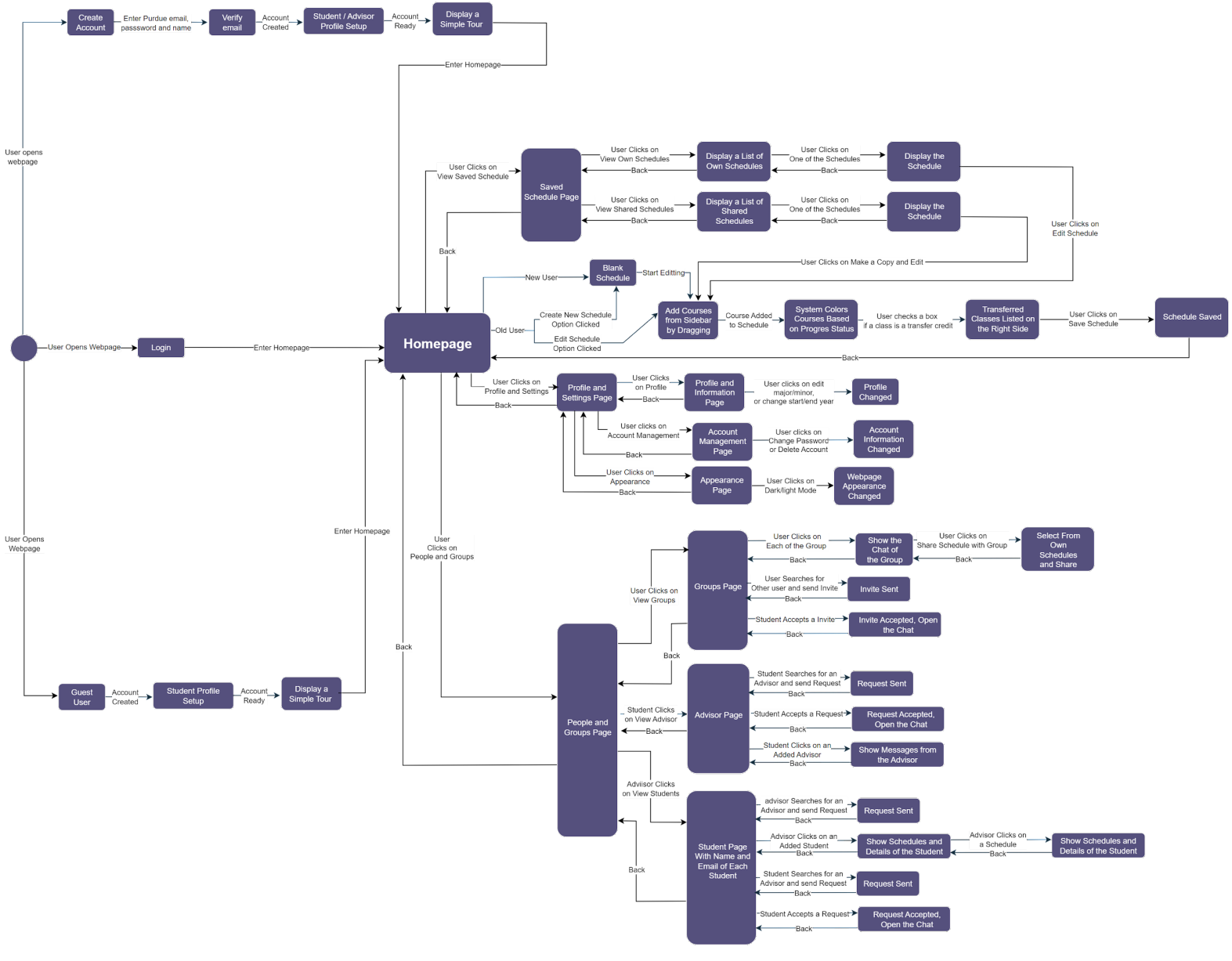
- **See shared schedules within groups**

  The below diagram shows the flow of data when the user clicks on a group to see shared schedules. First the request to see the schedules gets sent to the server, the server then query the database for requested schedules. Then, the server returns the schedule retrieved from the database, and once the client receives the data, the schedules get displayed on the UI for the user to see.



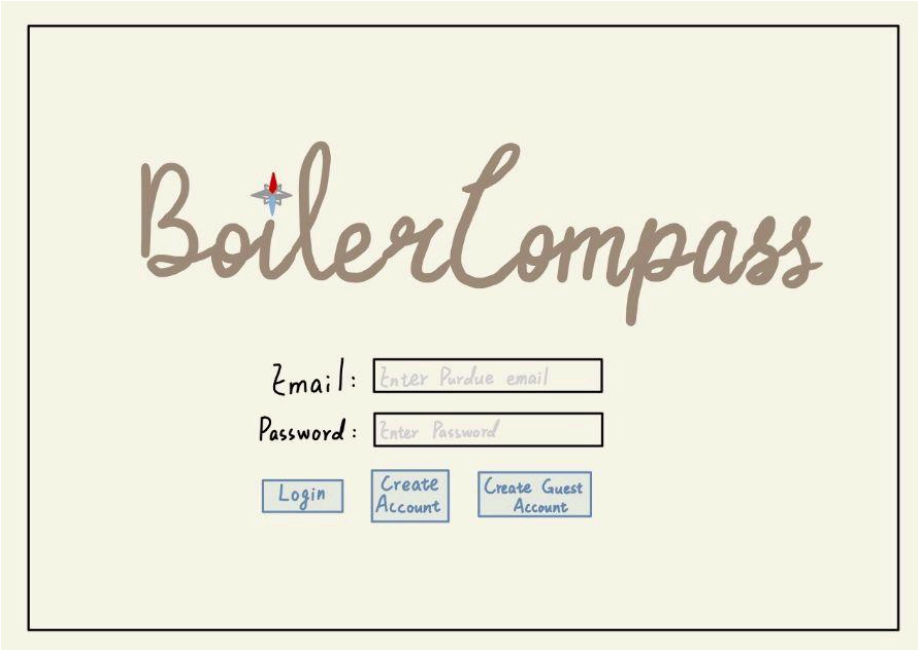
Overview Navigation Flow Diagram

The navigation flow diagram below shows the overall sequence of events and stages. When the user enters the webpage, he/she can choose to login, create an account, or continue as a guest user. The user can access functions by clicking on icons and links on the homepage, most pages allow users to go back to the previous page.



UI Mockup

Login Page:



Homepage:

